

Paytm PML — Next-Round Take-Home

Take-Home Review

August 11, 2026

CONTENTS

Paytm PML — Next-Round Take-Home	
Wallet / P2P Transfer Service (Deploy & Reason)	
What to build — a wallet / peer-to-peer transfer service	
The thing we will probe live (your correctness gate)	
Two design calls we specifically want you to reason about (in the write-up)	
The real focus — deploy, containerize, observe	
What to send back	
What we're evaluating (and not)	

Paytm PML — Next-Round Take-Home

Wallet / P2P Transfer Service (Deploy & Reason)

Congratulations on clearing the earlier rounds. This next step is a bigger, hands-on exercise: **build it, deploy it, and reason about it**. The surface is deliberately small — a wallet that lets people hold a balance and transfer to each other — but money movement under concurrency and retries is unforgiving, and that's exactly what we want to see: the simplest correct design, your reasoning about the trade-offs, and whether you can deploy and operate what you build.

Budget about a day of focused work (up to ~1.5 if you want to go deep). Everything runs on free tiers — please spend ₹0. Use AI tools if you like; just tell us in the write-up what you directed vs. decided.

What to build — a wallet / peer-to-peer transfer service

Users hold a balance (integer paise) and transfer to each other. You must never lose or create money, never double-apply a retried transfer, and never let one user move another user's money.

Core API (keep it small):

1. **Auth** — users authenticate with a signed token (JWT or similar). The caller's identity comes from the **verified token — never a trusted header or body field**.
2. `POST /accounts` → get-or-create the caller's wallet → `{ balance }`. Idempotent: calling twice returns the same wallet, never two.
3. `POST /transfers { to_user, amount_paise, idempotency_key }` → move funds from the caller to `to_user`; if `to_user` has no wallet yet, create it as part of the transfer. Succeed only if the caller has sufficient balance, else reject (a 4xx you pick and defend). Returns `{ transfer_id, new_balance }`.
4. `GET /accounts/me` → `{ balance }`; `GET /transfers/{id}` → the transfer's details (only a participant may read it).
5. `GET /healthz` (liveness) and `GET /readyz` (readiness incl. the datastore).

Invariants (this is the real exercise):

- **Conservation.** Total money is conserved across every transfer — no partial/torn transfers, no lost or created paise, even under a concurrent burst.

- **Idempotency.** Every transfer carries an `idempotency_key`; a retry with the same key returns the original outcome and does **not** move funds twice. Same key + a different body → 409.
- **Get-or-create is race-free.** Two concurrent first-transfers between the same brand-new pair must create each wallet **exactly once** — never two wallet rows, never a 500. (This is the crux.)
- **Authorization.** A caller can only spend from their own wallet and only read transfers they're part of — identity from the token, never a client-supplied id.
- **Money as integer paise** (never float).

Persistence: a real database (Postgres recommended; not an in-memory map), schema via migrations.

The thing we will probe live (your correctness gate)

Create two brand-new users. Fire many concurrent first-transfers between them at once, and many concurrent retries of the same transfer with the same `idempotency_key`. Exactly the right amount moves, each wallet is created exactly once, retries apply once, balances reconcile — never overspent, never a 500. **Give us a one-command way (a script or curl one-liner) to fire that burst at your live service.**

Two design calls we specifically want you to reason about (in the write-up)

1. **The get-or-create + transfer race.** There's a simple, correct way to make "create the counterparty wallet if absent, then move funds" safe under concurrency, and heavier ways. Pick the simplest correct mechanism and defend it — how do you avoid the two-concurrent-first-requests-both-insert race (the classic find-or-create failure that 500s), and where exactly does the idempotency key sit relative to the balance mutation?
 2. **Consistency vs. availability.** When the database is slow or unavailable, what does the transfer path do — reject (favor consistency: never double-spend) or degrade? And the read path (GET balance) under load? State your NFR priorities explicitly and justify them for a money workload — don't just assert one.
-

The real focus — deploy, containerize, observe

1. **Containerize properly** — multi-stage Dockerfile → small non-root image, container HEALTHCHECK; a `docker-compose.yml` that brings up app + db in one command; deploy the container image (not a buildpack); 12-factor config (everything sensitive via environment, nothing committed). Confirm it builds and its tests pass from a clean checkout before you send it.
 2. **Deploy it live** — a public URL on a free host (Render / Railway / Fly.io / Koyeb), backed by free managed Postgres (Neon / Supabase / Railway). It should just work when we hit it.
 3. **Make it observable** — a public link to view the logs. Structured JSON logs with a correlation id per request, logging the meaningful events (transfer applied, insufficient-funds rejected, idempotent replay, the concurrent request that lost the get-or-create race, auth failures). Basic metrics (request count, latency incl. p99, error rate, transfers applied/rejected) via `/metrics` and/or a dashboard.
-

What to send back

- The live app URL, the public GitHub repo, and the public logs link.
 - The burst script (concurrent transfers + retries) so we can re-run your gate.
 - A one-page write-up we'll read closely:
 - your data model;
 - the mechanism that makes get-or-create + transfer safe under concurrency and why it's the simplest correct one (name the heavier alternatives you rejected);
 - how idempotency keys are stored / matched / expired and same-key-different-body detected;
 - how identity/authorization works (token → caller, no trusted header);
 - your consistency-vs-availability decision;
 - the edge cases you handled (insufficient funds, self-transfer, unknown recipient, replay vs. conflict, negative/zero amount);
 - your containerization / deploy / observability choices;
 - your AI usage (directed vs. decided);
 - and a short cost note showing it's all free-tier.
-

What we're evaluating (and not)

Money conservation + correct idempotency under concurrency (we'll reproduce it against your live URL); a race-free get-or-create; token-based authorization (no trusted header); the quality of your NFR reasoning; and that it's genuinely deployed, containerized, and observable.

We are **not** grading UI, feature breadth, or exotic auth — keep the surface tiny and the reasoning sharp. **A small service that never loses money and reasons honestly about its trade-offs beats a large, over-engineered one.**

Looking forward to seeing it live.

— Jithesh, Engineering, Paytm Money